Correlation-Aware Machine Learning Task Scheduling for Power Efficiency and GPU Provisioning in AI Data Centers

Srinivasan Subramaniyan

Power Aware Computing Systems (PACS) Lab
Advisor: Prof. Xiaorui Wang

March 16, 2026



The Ohio State University

Overview

1. Introduction
2. Correlation Analysis
3. Thesis Contributions
4. CorrGPU
5. CATS
6. Conclusion

Machine Learning as a Service (MLaaS)

Why MLaaS?

- Many users cannot afford or maintain high-end GPUs.

Machine Learning as a Service (MLaaS)

Why MLaaS?

- Many users cannot afford or maintain high-end GPUs.
- They rely on cloud vendors to provide GPUs on demand for AI/ML workloads.



Machine Learning as a Service (MLaaS)

Why MLaaS?

- Many users cannot afford or maintain high-end GPUs.
- They rely on cloud vendors to provide GPUs on demand for AI/ML workloads.



What runs on these clouds?

- Users execute ML frameworks such as TensorFlow, PyTorch, and MXNet for image classification, speech recognition, and NLP.
- The rapidly growing demand for MLaaS is driving the construction of large GPU data centers.



Challenges in GPU Data Centers

Key Challenges for Data Center Operators

- GPU clusters are expensive to build, so operators must decide: **How many GPUs should be purchased?**
- Over-provisioning leads to wasted resources and unnecessarily high capital expense (CapEx).
- Under-provisioning increases contention and may delay ML training jobs.

Challenges in GPU Data Centers

Key Challenges for Data Center Operators

- GPU clusters are expensive to build, so operators must decide: **How many GPUs should be purchased?**
- Over-provisioning leads to wasted resources and unnecessarily high capital expense (CapEx).
- Under-provisioning increases contention and may delay ML training jobs.

After Deployment

- GPUs are also power hungry, and large clusters consume substantial energy.
- Operators must optimize runtime efficiency to reduce operational cost (OpEx).

Challenges in GPU Data Centers

Key Challenges for Data Center Operators

- GPU clusters are expensive to build, so operators must decide: **How many GPUs should be purchased?**
- Over-provisioning leads to wasted resources and unnecessarily high capital expense (CapEx).
- Under-provisioning increases contention and may delay ML training jobs.

After Deployment

- GPUs are also power hungry, and large clusters consume substantial energy.
- Operators must optimize runtime efficiency to reduce operational cost (OpEx).

Two fundamental questions: Purchase efficiently, then operate efficiently.

Research Problem

How can we improve the efficiency of GPU data centers while maintaining ML performance?

Research Goals

- Determine the minimum number of GPUs needed to serve ML workloads in a timely manner.
- Reduce the power consumption of deployed GPU clusters.
- Maintain acceptable ML training performance and job completion time.

Research Problem

How can we improve the efficiency of GPU data centers while maintaining ML performance?

Research Goals

- Determine the minimum number of GPUs needed to serve ML workloads in a timely manner.
- Reduce the power consumption of deployed GPU clusters.
- Maintain acceptable ML training performance and job completion time.

Thesis Perspective

- **Before deployment:** minimize GPU purchase cost.
- **After deployment:** optimize GPU power consumption.

Key Observation

Limitations of Existing Scheduling Approaches

- Many existing approaches assume that GPU utilization of ML tasks is constant (e.g., peak or average).
- However, real ML workloads exhibit **significant fluctuations** in GPU utilization over time.

Key Observation

Limitations of Existing Scheduling Approaches

- Many existing approaches assume that GPU utilization of ML tasks is constant (e.g., peak or average).
- However, real ML workloads exhibit **significant fluctuations** in GPU utilization over time.

Example from Production Traces

- Analysis of real-world traces (e.g., Microsoft Philly) shows that ML workloads rarely peak at the same time.
- Many tasks exhibit **weak or negative correlations**.

Key Observation

Limitations of Existing Scheduling Approaches

- Many existing approaches assume that GPU utilization of ML tasks is constant (e.g., peak or average).
- However, real ML workloads exhibit **significant fluctuations** in GPU utilization over time.

Example from Production Traces

- Analysis of real-world traces (e.g., Microsoft Philly) shows that ML workloads rarely peak at the same time.
- Many tasks exhibit **weak or negative correlations**.

Opportunity

- Correlation-aware scheduling can improve GPU utilization.
- This can reduce both **GPU provisioning cost** and **power consumption**.

Correlation Analysis

Pearson Correlation Coefficient

The correlation between two samples $x = \{x_1, \ldots, x_n\}$ and $y = \{y_1, \ldots, y_n\}$ is defined as:

$$\rho(x, y) = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

- Correlation measures how two variables move together.

Correlation Analysis

Pearson Correlation Coefficient

The correlation between two samples $x = \{x_1, \ldots, x_n\}$ and $y = \{y_1, \ldots, y_n\}$ is defined as:

$$\rho(x, y) = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

- Correlation measures how two variables move together.
- Range: $-1 \leq \rho \leq 1$

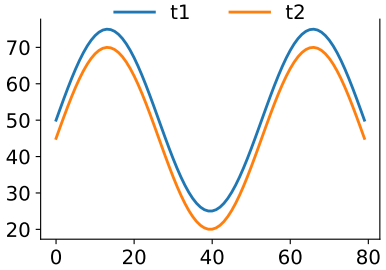
Correlation Analysis

Consider two utilization distributions: t_1 and t_2.

Correlation Analysis

Consider two utilization distributions: t_1 and t_2.

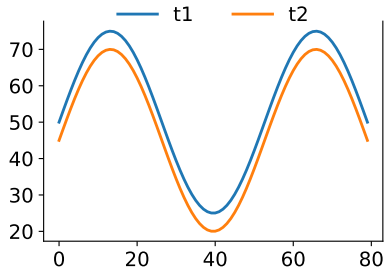
- **Positive Correlation**



Correlation Analysis

Consider two utilization distributions: t_1 and t_2.

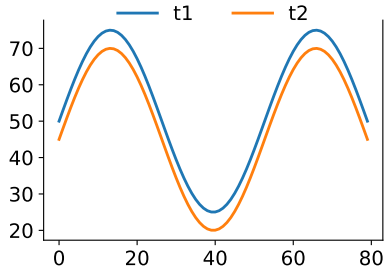
- **Positive Correlation**
- Both distributions increase or decrease together.



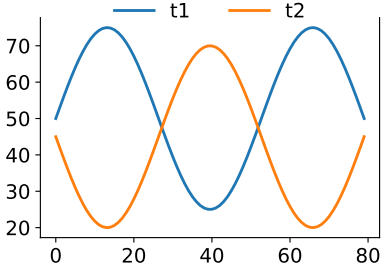
Correlation Analysis

Consider two utilization distributions: t_1 and t_2.

- **Positive Correlation**
- Both distributions increase or decrease together.



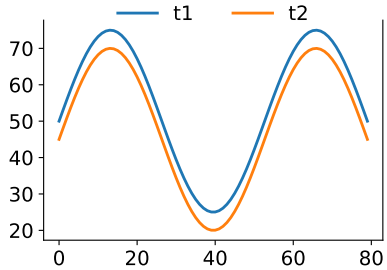
- **Negative Correlation**



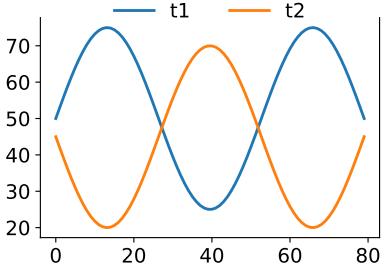
Correlation Analysis

Consider two utilization distributions: t_1 and t_2.

- **Positive Correlation**
- Both distributions increase or decrease together.



- **Negative Correlation**
- When one distribution increases, the other decreases.



Correlation Analysis of ML Workloads

Workload Setup:

- **Workloads:** Vision (DenseNet, Inception, MobileNet, ResNet, GoogleNet, VGG) and Transformer (Swin, GPT-2).
- **Configuration:** 80 epochs, batch size $B \in \{10, 20, 30, 40, 50, 60\}$.

Correlation Analysis of ML Workloads

- **Workloads:** Vision (DenseNet, Inception, MobileNet, ResNet, GoogleNet, VGG) and Transformer (Swin, GPT-2).
- **Configuration:** 80 epochs, batch size $B \in \{10, 20, 30, 40, 50, 60\}$.

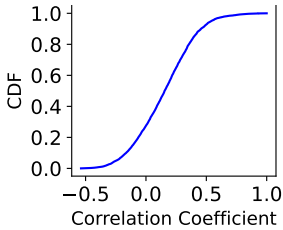


Figure 1: CDF

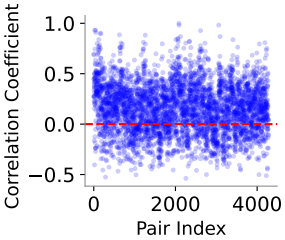


Figure 2: Scatter Plot

Observation:
Most workload pairs exhibit **weak or negative correlations**, indicating that GPU utilization does not peak at the **same time**.

Correlation Analysis on ML Traces

Trace-Based Analysis Setup

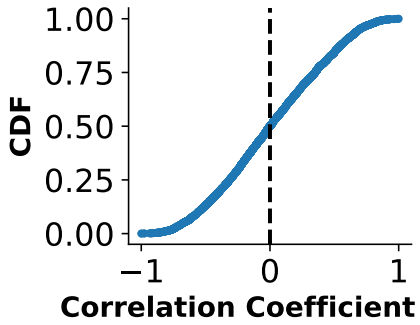
- **Philly trace:** Production ML training tasks from Microsoft's GPU data center.
- **Cluster:** Primarily Nvidia P100 GPUs.
- **Sampled tasks:** 600 tasks selected from the middle of the trace.
- **Metric:** Aggregate GPU utilization per job across all assigned GPUs.

Goal

- Quantify similarity between task utilization patterns.
- Determine whether ML tasks peak at the same time.

Correlation Analysis on ML Traces

Correlation Distribution from the Philly Trace

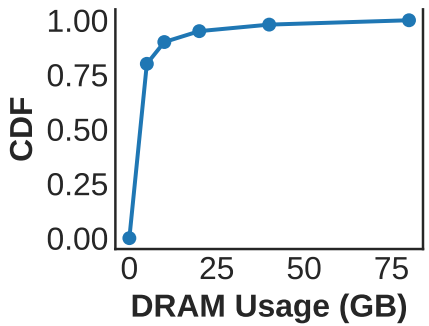


- 46% of task pairs show **negative correlation**.
- 34% fall between 0 and 0.3 (weak positive correlation).
- About 80% satisfy $|r_{xy}| \leq 0.3$.

Observation 1: The GPU utilization of most training tasks is loosely correlated, so they usually do not peak at the same time.

Correlation Analysis on ML Traces

GPU Memory Utilization Across Workloads



- Most workloads consume significantly less than **16 GB**.
- Many workloads also require less than **32 GB**.
- GPU memory usage is fragmented across workloads.

Observation 2: GPU memory consumption is fragmented, providing an opportunity to consolidate tasks.

Correlation Analysis on ML Traces

Overhead of Correlation Computation

Samples	1K	100K	1000K
Time (ms)	0.295	1.325	12.999

- Even with 1 million samples, correlation computation takes less than **13 ms**.
- Correlation analysis introduces **negligible overhead** .

Takeaway: Correlation is a lightweight signal that can guide **online GPU task consolidation**.

Thesis Contributions

Goal: Improve Efficiency of GPU Data Centers

Contribution 1: CorrGPU [IPCCC 2025]

- A correlation-aware scheduling algorithm to minimize the number of GPUs required in a data center.
- Reduces capital expense (CapEx) by efficiently provisioning GPUs for ML workloads.

Thesis Contributions

Goal: Improve Efficiency of GPU Data Centers

Contribution 1: CorrGPU [IPCCC 2025]

- A correlation-aware scheduling algorithm to minimize the number of GPUs required in a data center.
- Reduces capital expense (CapEx) by efficiently provisioning GPUs for ML workloads.

Contribution 2: CATS [ICS 2026]

- A correlation-aware scheduling framework for reducing GPU power consumption.
- Consolidates ML workloads and dynamically adjusts GPU frequency.

Thesis Contributions

Goal: Improve Efficiency of GPU Data Centers

Contribution 1: CorrGPU [IPCCC 2025]

- A correlation-aware scheduling algorithm to minimize the number of GPUs required in a data center.
- Reduces capital expense (CapEx) by efficiently provisioning GPUs for ML workloads.

Contribution 2: CATS [ICS 2026]

- A correlation-aware scheduling framework for reducing GPU power consumption.
- Consolidates ML workloads and dynamically adjusts GPU frequency.

Overall Impact

- Improve GPU utilization.
- Reduce GPU infrastructure cost.
- Lower power consumption of GPU clusters.

CorrGPU

Exploiting ML Task Correlation in the Minimization of Capital Expense for GPU Data Centers

Optimization Formulation

Goal: Minimize GPU Infrastructure Cost

- Determine the **minimum number of GPUs** required to serve all ML tasks.
- Fewer GPUs directly reduce the **capital expense (CapEx)** of the data center.

Optimization Formulation

CapEx Model

$$\text{CapEx} = \left(\sum_{j=1}^{G} y_j \right) \times P$$

- G: total number of candidate GPUs
- y_j: binary variable indicating whether GPU j is active
- P: price per GPU

Optimization Formulation

Optimization Objective

$$\min \sum_{j=1}^{G} y_j$$

- Minimize the number of active GPUs.
- This directly minimizes **CapEx**.

Detailed formulation appears in **Chapter 3 of the thesis**.

Iterative Optimization Approach

Finding the Minimum Number of GPUs

1. Initialize $G = 1$.

Iterative Optimization Approach

Finding the Minimum Number of GPUs

1. Initialize $G = 1$.
2. Solve the optimization problem:

$$\min \sum_{j=1}^{G} y_j$$

subject to:

Iterative Optimization Approach

Finding the Minimum Number of GPUs

1. Initialize $G = 1$.
2. Solve the optimization problem:

$$\min \sum_{j=1}^{G} y_j$$

subject to:

- Memory capacity constraints
- Correlation constraints
- Task assignment constraints
- Dispatching constraints

Iterative Optimization Approach

Finding the Minimum Number of GPUs

1. Initialize $G = 1$.
2. Solve the optimization problem:

$$\min \sum_{j=1}^{G} y_j$$

subject to:

- Memory capacity constraints
- Correlation constraints
- Task assignment constraints
- Dispatching constraints

3. If the solver finds no feasible solution, increase G.

Iterative Optimization Approach

Finding the Minimum Number of GPUs

1. Initialize $G = 1$.
2. Solve the optimization problem:

$$\min \sum_{j=1}^{G} y_j$$

subject to:

- Memory capacity constraints
- Correlation constraints
- Task assignment constraints
- Dispatching constraints

3. If the solver finds no feasible solution, increase G.
4. The first feasible solution gives the minimum number of GPUs G^*.

$$G^* = \min\{G \mid \text{all constraints satisfied}\}$$

Computational Complexity and Solver Limitations

- We use the PuLP optimization solver to compute optimal solutions.
- Solver tolerance is set to 1×10^{-6}.

Tasks	2	4	8	16	32	64	128
Solve Time (s)	0.04	0.07	0.16	0.22	4.92	8.95	80.9

- The number of constraints grows rapidly as the number of tasks increases.
- Solver runtime becomes impractical for large traces.

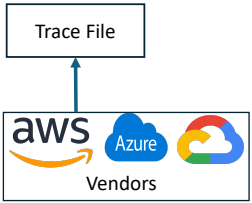
Motivation: Design a lightweight heuristic scheduling algorithm (CorrGPU).

CorrGPU: Lightweight Heuristic Algorithm

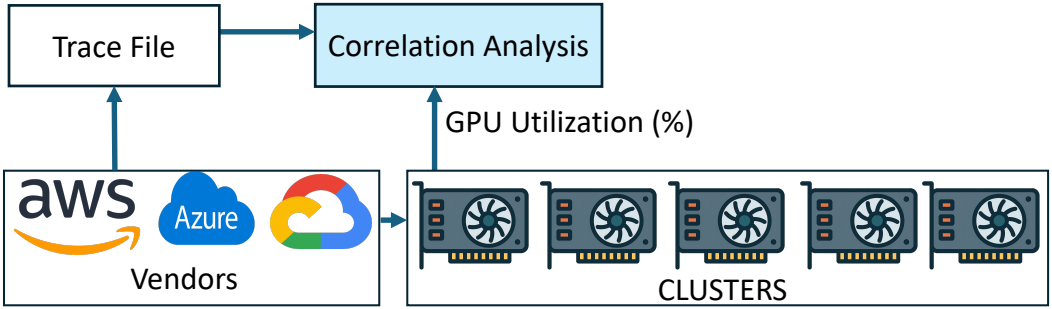
CorrGPU consists of two key components:

- Correlation Analysis.
- Dispatching.

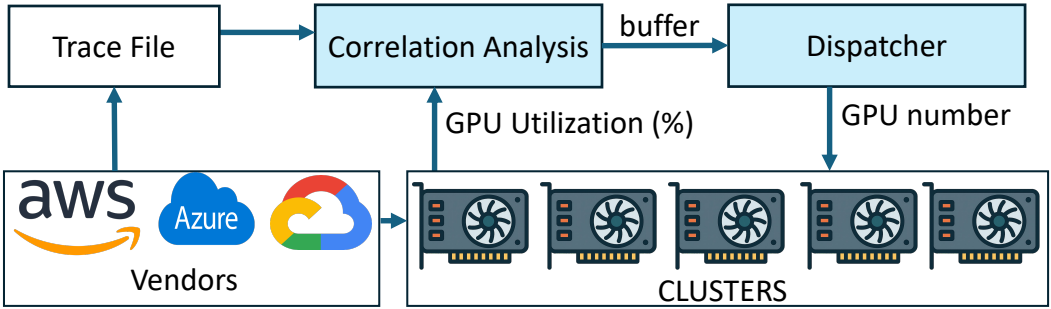
Framework Design



Framework Design



Framework Design



For the detailed steps, please refer to Algorithm 1 in Section III–C in the paper.

Example

- Two tasks : (t_1) and (t_2)
- Average GPU utilization: 52.3% (t_1), 42.7% (t_2)
- Initial sampled utilization: $t_1 = 40\%$, $t_2 = 60\%$

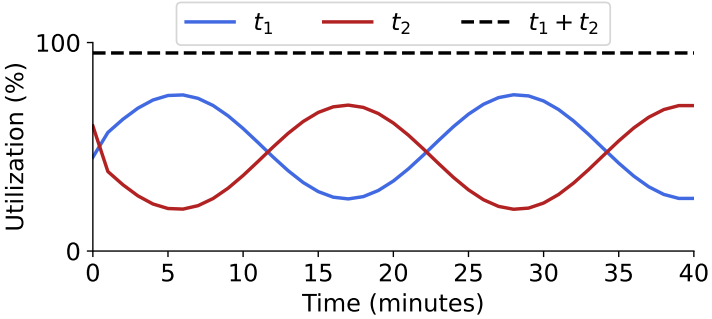


Figure 3: GPU Utilization of ResNet and SqueezeNet

Example

Baseline (First):

- Schedules tasks using initial utilization in the first second.
- Aggregated Initial Utilizations: $40\% + 60\% = 100\%$.
- Hence, $t_1 \rightarrow \text{GPU}_0$, $t_2 \rightarrow \text{GPU}_1$.

Example

Baseline (First):

- Schedules tasks using initial utilization in the first second.
- Aggregated Initial Utilizations: $40\% + 60\% = 100\%$.
- Hence, $t_1 \rightarrow \text{GPU}_0$, $t_2 \rightarrow \text{GPU}_1$.

CorrGPU:

- Correlation coefficient $\rho = -1$ (negative correlation).
- Aggregated utilization: $52.3\% + 42.7\% = 95.0\% < 100\%$.
- Hence $t_1, t_2 \rightarrow \text{GPU}_0$.

Example

Baseline (First):

- Schedules tasks using initial utilization in the first second.
- Aggregated Initial Utilizations: $40\% + 60\% = 100\%$.
- Hence, $t_1 \rightarrow \text{GPU}_0$, $t_2 \rightarrow \text{GPU}_1$.

CorrGPU:

- Correlation coefficient $\rho = -1$ (negative correlation).
- Aggregated utilization: $52.3\% + 42.7\% = 95.0\% < 100\%$.
- Hence $t_1, t_2 \rightarrow \text{GPU}_0$.

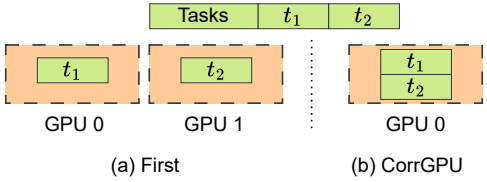


Figure 3: CorrGPU uses only one GPU.

Example

Baseline (First):

- Schedules tasks using initial utilization in the first second.
- Aggregated Initial Utilizations: $40\% + 60\% = 100\%$.
- Hence, $t_1 \rightarrow \text{GPU}_0$, $t_2 \rightarrow \text{GPU}_1$.

CorrGPU:

- Correlation coefficient $\rho = -1$ (negative correlation).
- Aggregated utilization: $52.3\% + 42.7\% = 95.0\% < 100\%$.
- Hence $t_1, t_2 \rightarrow \text{GPU}_0$.

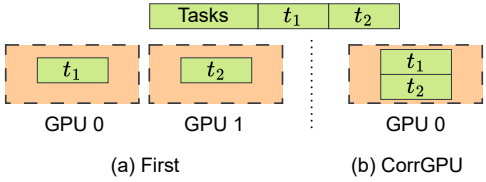


Figure 3: CorrGPU uses only one GPU.

Solution	First	CorrGPU
CapEx	5.0	2.5

Table 1: CapEx comparison (A-100 GPU).

Experimental Setup

- **Real-world Trace:** Alibaba PAI trace.
- **Hardware Testbed:** Six NVIDIA V100 GPUs.
- **Spatial Sharing:** Enabled via CUDA Multiprocessing Service (MPS).
- **Trace-driven Simulation:** Implemented using a Python simulator based on `GPUColo`.
- **Baselines:**
  1. **Harmony:** Uses peak utilization.
  2. **First:** Uses first (initial) utilization.

Hardware Evaluation

- Selected **14 ML tasks** from the PAI trace with diverse resource demands.
- Each task is characterized by:
  - Arrival time and duration,
  - Memory consumption,
  - Average GPU utilization.

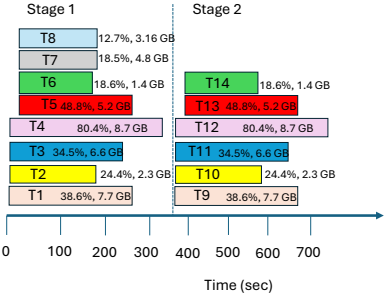


Figure 4: Arrival time, duration, memory usage, and GPU utilization for 14 real-world ML tasks.

Hardware Evaluation

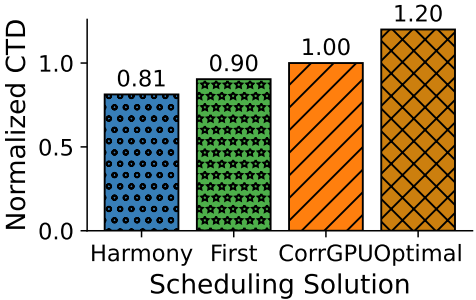


Figure 4: Cumulative Task Duration.

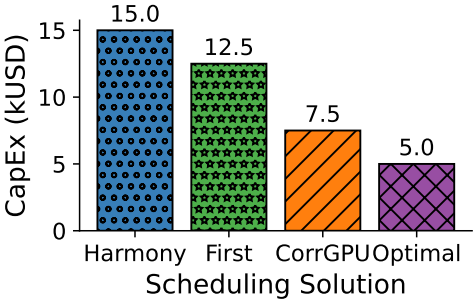


Figure 5: Capital Expenditure.

Simulation Evaluation

- **PAI trace** consisting of **5000 tasks**.
- CorrGPU estimates the minimum GPUs required while maintaining acceptable *CTD*.
- On average:
  - Harmony uses 23.2 GPUs,
  - First uses 16.7 GPUs,
  - CorrGPU requires only 15.32 GPUs.

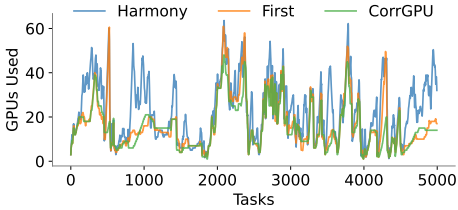


Figure 6: CorrGPU uses fewer GPUs than baselines in large-scale simulation using the PAI trace.

Simulation Evaluation

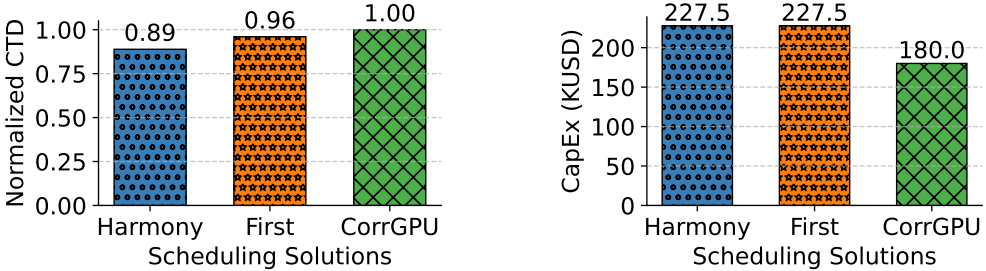


Figure 6: **Simulation results:** (a) CorrGPU maintains acceptable CTD within due times while (b) achieving significantly lower CapEx than both baselines.

CATS

Correlation-aware Task Scheduling for GPU Power Optimization in AI Data Centers

Power and Performance Model

Goal: Minimize Data Center Power Consumption

- Reduce the **total power consumption** of GPU data centers.
- Consolidate ML tasks onto fewer GPUs and servers.
- Adjust GPU frequency while keeping training slowdown within the SLA.

Power and Performance Model

GPU Power Model

$$P_{\text{gpu}} = P_{gs} + \alpha f$$

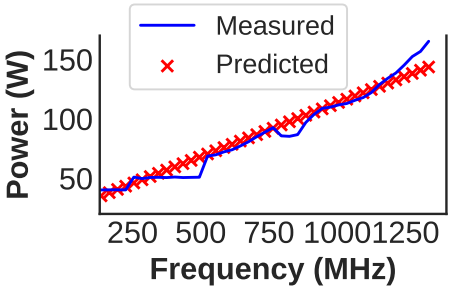
- P_{gs}: static GPU power
- α: dynamic power coefficient
- f: GPU operating frequency

$$P_{\text{gpu}} = 23.3 + 0.09f$$

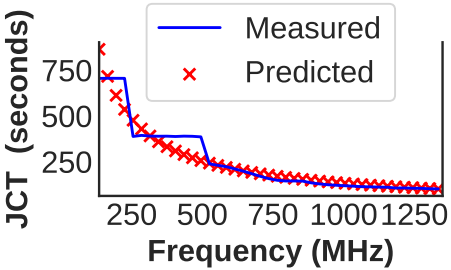
- Measured on NVIDIA V100 GPUs
- Model accuracy: $R^2 \approx 0.95$

Power and Performance Model

Power and Performance Trade-off



(a) Power consumption



(b) Job completion time

- Lower frequency reduces GPU power.
- However, it increases job completion time (JCT).

Optimization Formulation

Goal: Minimize Total Data Center Power

- Consolidate ML tasks onto fewer GPUs.
- Turn off unused GPUs and servers to save power.
- Dynamically scale GPU frequency without violating task SLAs.

Optimization Formulation

Server Power Model

$$P_{\text{server},i} = \begin{cases} P_i^{static} + \sum_{j=1}^{w_i} P_{\text{gpu},ij}(f_{ij}) + P_{other}, & z_i = 1 \\ 0, & z_i = 0 \end{cases}$$

- P_i^{static}: server baseline power
- $P_{\text{gpu},ij}$: GPU power consumption
- P_{other}: CPU, memory, and other system power

Optimization Formulation

Optimization Objective

$$\min \sum_{i=1}^{N} P_{\text{server},i}$$

subject to:

Detailed formulation appears in **Chapter 4 of the thesis**.

Optimization Formulation

Optimization Objective

$$\min \sum_{i=1}^{N} P_{\text{server},i}$$

subject to:

- GPU allocation constraints
- GPU memory capacity constraints
- GPU and server activation constraints
- GPU frequency bounds
- SLA (JCT) constraints

Detailed formulation appears in **Chapter 4 of the thesis**.

Computational Complexity and Solver Limitations

- The optimization problem is formulated as a MILP.
- Binary placement and activation variables make the problem NP-hard.

Computational Complexity and Solver Limitations

Solver Runtime

Tasks	2	4	8	16	32	64	128
Solve Time (s)	0.02	0.12	0.24	0.66	4.7	16.5	80.9

Computational Complexity and Solver Limitations

- Solver runtime grows rapidly with the number of tasks.
- Exact optimization becomes impractical for large workloads.

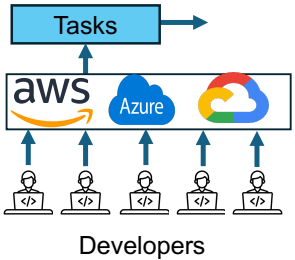
Motivation: Design a lightweight heuristic scheduling framework (CATS).

CATS: Lightweight Heuristic Framework

CATS consists of three key components:

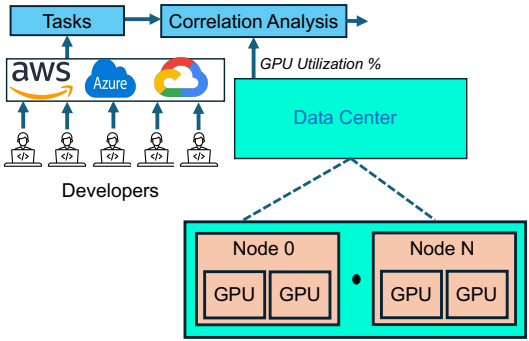
- Correlation Analysis
- Scheduler
- Power State Manager

Framework Design



Input: Incoming ML training tasks

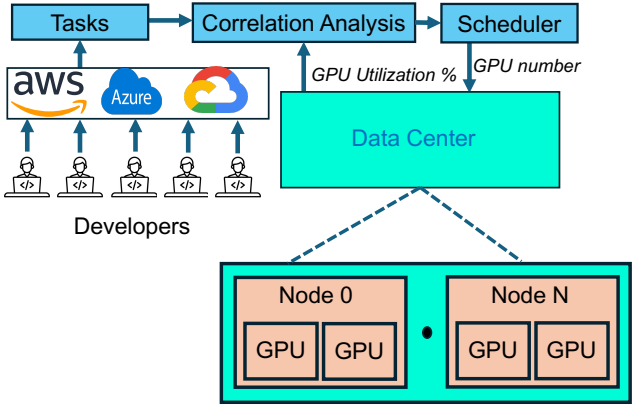
Framework Design



Correlation Analysis: Analyze task utilization correlation.

See Algorithm 1 in the paper.

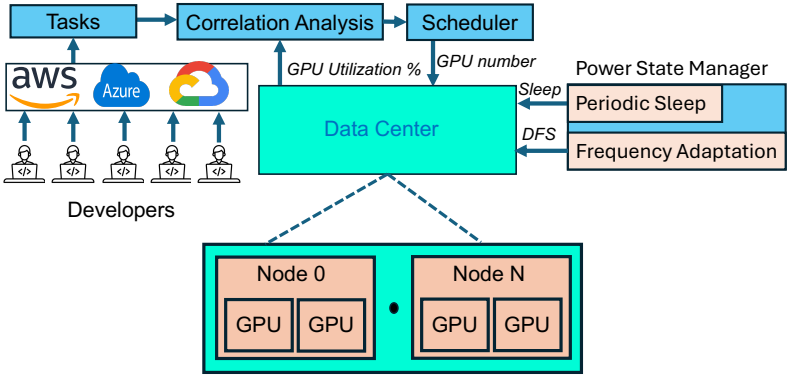
Framework Design



Scheduler: Consolidate tasks onto GPUs based on correlation and resource constraints.

See Algorithm 2 in the paper.

Framework Design

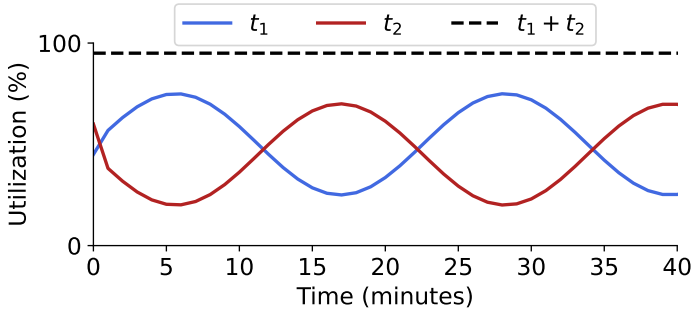


Power State Manager: Put unused GPUs/servers to sleep and adapt GPU frequency.

See Algorithms 3 and 4 in the paper.

Example

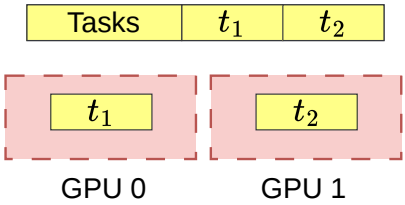
- Two ML training tasks: t_1 and t_2
- Memory demand: 12 GB (t_1), 10 GB (t_2)
- Average GPU utilization: 52.23% (t_1), 42.7% (t_2)
- Correlation coefficient: $\rho(t_1, t_2) = -1$



Example

Default Scheduling

- Each task is assigned to a separate GPU.
- $t_1 \rightarrow \text{GPU}_0$
- $t_2 \rightarrow \text{GPU}_1$
- Both GPUs remain active.



Default: one task per GPU.

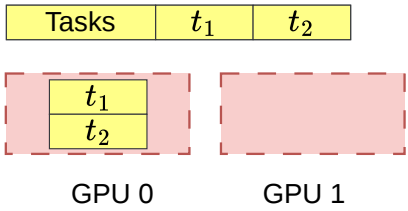
Example

CATS Scheduling

- Since $\rho(t_1, t_2) = -1$, the tasks do not peak at the same time.
- Total memory demand:

$$12\,\text{GB} + 10\,\text{GB} = 22\,\text{GB}$$

- Since $22\,\text{GB} < 32\,\text{GB}$, both tasks can share one GPU.

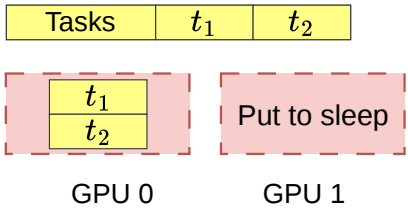


CATS checks correlation and memory feasibility.

Example

CATS Scheduling

- Assign both tasks to GPU_0.
- Put unused GPU_1 into sleep mode.
- Result: lower idle power consumption with only slight JCT impact.



CATS consolidates tasks and puts GPU_1 to sleep.

Experimental Setup

- **Hardware Testbed:** Single-node server with **six NVIDIA V100 GPUs**.
- **OS / Runtime:** Ubuntu 20.04 LTS, PyTorch 1.8.1.
- **GPU Frequency Range:** 135 MHz – 1350 MHz.
- **Spatial Sharing:** Enabled via CUDA Multiprocessing Service (MPS).
- **Thresholds:**
  - Correlation threshold = 0
  - Utilization threshold = 100%
- **Monitoring:** GPU utilization and power measured using `nvidia-smi`.

Workloads and Baselines

Workloads

- Workloads are derived from the **PAI trace**.
- Eight representative DNN workloads are used:
  - DenseNet
  - GoogleNet
  - ConvNeXt
  - ViT
  - Swin Transformer
  - ResNet
  - MobileNet
  - SqueezeNet
- These workloads provide diverse GPU utilization and memory demands.

Workloads and Baselines

Baselines

- **HotCloud:**
  - Uses **average utilization** for consolidation.
  - Consolidates tasks if combined average utilization is below threshold.
- **First:**
  - Uses **first-second utilization** for consolidation.
  - Consolidates tasks if initial aggregated utilization is below threshold.

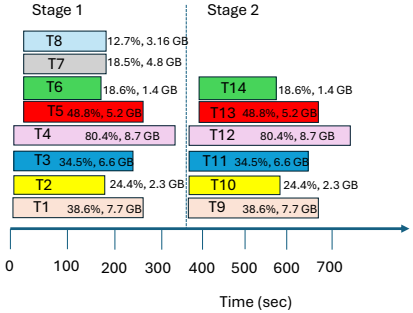
Workloads and Baselines

Large-Scale Evaluation

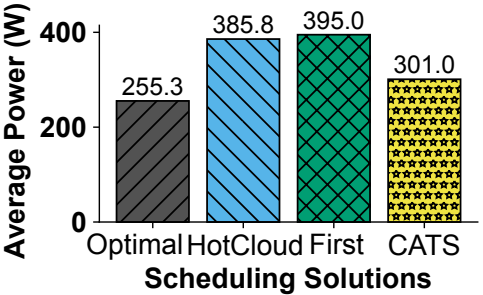
- Trace-driven simulator built from **Gavel** and **GPUColo**.
- Two production traces:
  - **PAI:** 5000 tasks
  - **Philly:** 2000 tasks
- Each server is equipped with **8 GPUs**.

Hardware Evaluation

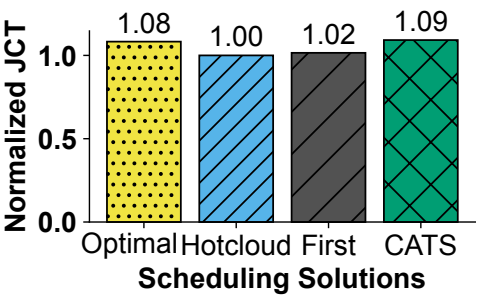
- We evaluate CATS using **14 ML tasks** sampled from the PAI trace.
- The experiment is divided into **two stages** to emulate bursty arrivals.
- Each task is characterized by:
  - Start time
  - GPU utilization
  - Memory consumption
- Task utilization predictions are perturbed with additive noise to test robustness.



Hardware Evaluation



(a) Average Power



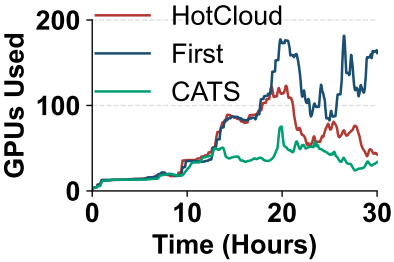
(b) Normalized JCT

- **Average power:** Optimal = 255.3 W, CATS = 301 W, HotCloud = 385.8 W, First = 395 W.
- CATS achieves substantially lower power than both baselines.
- Normalized JCT remains within the SLA threshold.

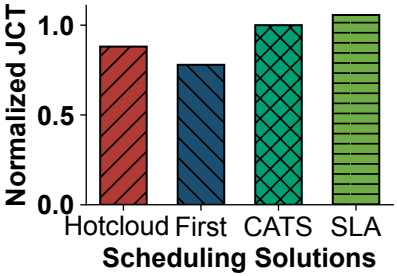
Simulation Evaluation: PAI Trace (1/2)

Setup

- **Trace:** 5000 tasks from the PAI trace.
- All GPUs and servers are initially asleep and are activated only when needed.
- Idle GPUs and servers are periodically put back to sleep for all methods.



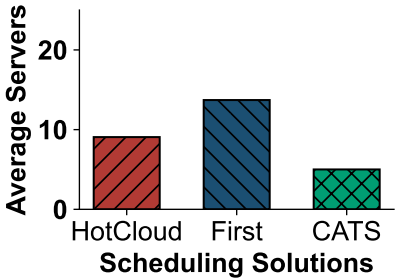
(a) Average GPUs used



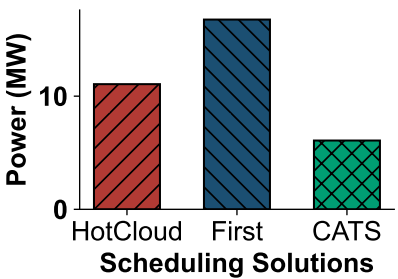
(b) Normalized JCT

Simulation Evaluation: PAI Trace (2/2)

Resource Usage and Power



(c) Average servers used



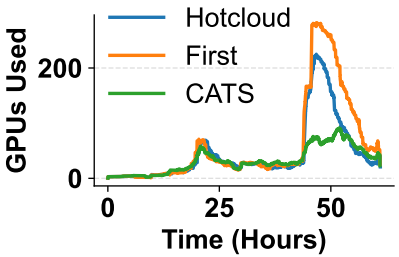
(d) Average power consumption

- CATS also uses the fewest servers.
- **Average power:** CATS = 6.08 MW.
- This is **63.59% lower** than First.

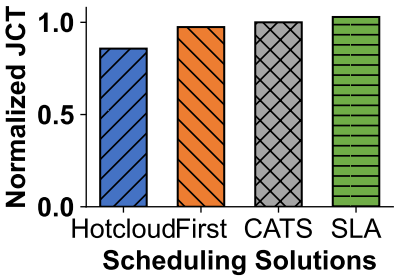
Simulation Evaluation: Philly Trace (1/2)

Setup

- **Trace:** 2000 tasks from the Philly trace.
- Same simulation setting as the PAI experiment.
- Goal: compare resource usage, JCT, and power consumption.



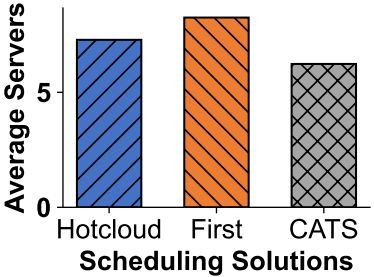
(a) Average GPUs used



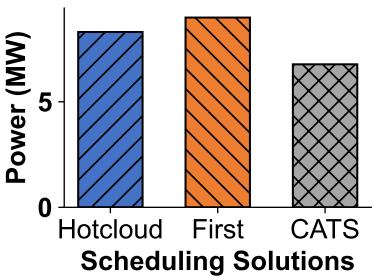
(b) Normalized JCT

Simulation Evaluation: Philly Trace (2/2)

Resource Usage and Power



(c) Average servers used



(d) Average power consumption

- CATS also uses fewer servers.
- **Average power:** CATS = 6.77 MW.
- Power reduction: 18.4% lower than HotCloud

Conclusion

- AI/ML growth has made **GPU cost** and **power efficiency** key challenges in modern data centers.

Conclusion

- AI/ML growth has made **GPU cost** and **power efficiency** key challenges in modern data centers.
- This thesis showed that ML workloads often have **weak or negative utilization correlations**, creating opportunities for efficient task consolidation.

Conclusion

- AI/ML growth has made **GPU cost** and **power efficiency** key challenges in modern data centers.
- This thesis showed that ML workloads often have **weak or negative utilization correlations**, creating opportunities for efficient task consolidation.
- We proposed two correlation-aware solutions:

Conclusion

- AI/ML growth has made **GPU cost** and **power efficiency** key challenges in modern data centers.
- This thesis showed that ML workloads often have **weak or negative utilization correlations**, creating opportunities for efficient task consolidation.
- We proposed two correlation-aware solutions:
  - **CorrGPU** for minimizing GPU provisioning cost (**CapEx**)

Conclusion

- AI/ML growth has made **GPU cost** and **power efficiency** key challenges in modern data centers.
- This thesis showed that ML workloads often have **weak or negative utilization correlations**, creating opportunities for efficient task consolidation.
- We proposed two correlation-aware solutions:
  - **CorrGPU** for minimizing GPU provisioning cost (**CapEx**)
  - **CATS** for minimizing runtime power consumption (**OpEx**)

Conclusion

- AI/ML growth has made **GPU cost** and **power efficiency** key challenges in modern data centers.
- This thesis showed that ML workloads often have **weak or negative utilization correlations**, creating opportunities for efficient task consolidation.
- We proposed two correlation-aware solutions:
  - **CorrGPU** for minimizing GPU provisioning cost (**CapEx**)
  - **CATS** for minimizing runtime power consumption (**OpEx**)
- Results show:

Conclusion

- AI/ML growth has made **GPU cost** and **power efficiency** key challenges in modern data centers.
- This thesis showed that ML workloads often have **weak or negative utilization correlations**, creating opportunities for efficient task consolidation.
- We proposed two correlation-aware solutions:
  - **CorrGPU** for minimizing GPU provisioning cost (**CapEx**)
  - **CATS** for minimizing runtime power consumption (**OpEx**)
- Results show:
  - Up to **20.88% lower CapEx** with CorrGPU

Conclusion

- AI/ML growth has made **GPU cost** and **power efficiency** key challenges in modern data centers.
- This thesis showed that ML workloads often have **weak or negative utilization correlations**, creating opportunities for efficient task consolidation.
- We proposed two correlation-aware solutions:
  - **CorrGPU** for minimizing GPU provisioning cost (**CapEx**)
  - **CATS** for minimizing runtime power consumption (**OpEx**)
- Results show:
  - Up to **20.88% lower CapEx** with CorrGPU
  - Up to **24.7% lower power** with CATS

Conclusion

- AI/ML growth has made **GPU cost** and **power efficiency** key challenges in modern data centers.
- This thesis showed that ML workloads often have **weak or negative utilization correlations**, creating opportunities for efficient task consolidation.
- We proposed two correlation-aware solutions:
  - **CorrGPU** for minimizing GPU provisioning cost (**CapEx**)
  - **CATS** for minimizing runtime power consumption (**OpEx**)
- Results show:
  - Up to **20.88% lower CapEx** with CorrGPU
  - Up to **24.7% lower power** with CATS
- Correlation-aware scheduling provides a practical path toward **more efficient GPU data centers**.

Thank You for Your Attention!

Questions?

This work is supported, in part, by the U.S. NSF under award CNS-2336886.

Backup: GPU Utilization Prediction

Utilization Prediction in Practice

- Correlation analysis relies on GPU utilization traces of ML tasks.
- In practice, utilization values are obtained from:
  - historical execution traces, or
  - runtime monitoring tools such as `NVML` or `nvidia-smi`.
- In our hardware evaluation, we emulate prediction uncertainty by generating utilization traces from a **Gaussian distribution** whose mean and variance match those observed in real ML workloads.
- **Additive noise** is injected into the traces to represent prediction error.
- Even under noisy predictions, the correlation-aware scheduler remains effective because consolidation decisions rely on **coarse-grained utilization trends**.

Backup: Discussion and Extensions

Applicability to Emerging Workloads

- Modern AI workloads such as large language model (LLM) training often exhibit long execution times and varying GPU utilization patterns.
- Correlation-aware scheduling can help identify workloads whose utilization peaks do not occur simultaneously, enabling efficient task consolidation.

Multi-Dimensional Resource Considerations

- In this work, task consolidation primarily considers:
  - GPU utilization correlation
  - GPU memory capacity
- Future extensions could incorporate additional resource dimensions, such as memory bandwidth or CPU usage, when making consolidation decisions.